

CAMPUS CÁMARA COMERCIO SEVILLA

# MEMORIA TÉCNICA Y DESPLIEGUE EN DOCKER

Plataforma de Reservas Culturales de Sevilla

---

proyecto-reservas

| Dato                     | Valor                                                                              |
|--------------------------|------------------------------------------------------------------------------------|
| Ciclo / Módulo           | DAW - Despliegue de aplicaciones web                                               |
| Resultado de aprendizaje | RA 1: Implanta arquitecturas web analizando y aplicando criterios de funcionalidad |
| Periodo de la actividad  | 25/09/2026 - 02/10/2026                                                            |
| Alumno/a                 | Adrián López Atalaya                                                               |
| Fecha de entrega         | 02/10/2026                                                                         |

# 1. Introducción y arquitectura del sistema

## 1.1 Objetivo

Una empresa de actividades culturales de Sevilla necesita publicar una aplicación web para gestionar reservas de talleres, visitas y eventos. El equipo de desarrollo entrega una API, una interfaz web y un esquema de base de datos. El **equipo de despliegue** (este trabajo) debe convertir esa entrega en un servicio operativo, mantenible y documentado, ejecutable en Docker Desktop con un único comando:

```
docker compose up --build
```

## 1.2 Componentes

La infraestructura se compone de **tres servicios** aislados, una **red privada** y un **volumen** de datos. Se comunican dentro de la red virtual `reservas-network`.

| Elemento                     | Contenedor / nombre | Tecnología                      | Puerto | ¿Publicado en el host? | Responsabilidad                                                                                                         |
|------------------------------|---------------------|---------------------------------|--------|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Servicio web (proxy inverso) | reservas_web        | Nginx                           | 80     | Sí (80:80)             | Único punto de entrada. Recibe el HTTP del cliente, sirve el contenido estático y redirige <code>/api/</code> a la API. |
| Servicio API                 | reservas_api        | Node.js 20 / Express            | 3000   | No                     | Lógica de negocio. Consulta y escribe en la base de datos.                                                              |
| Servicio base de datos       | reservas_db         | MySQL 8.0                       | 3306   | No                     | Almacena eventos y reservas. Tiene healthcheck.                                                                         |
| Red privada                  | reservas-network    | Red bridge de Docker            | -      | -                      | Aísla la comunicación entre los tres servicios.                                                                         |
| Volumen                      | mysql_data          | Volumen administrado por Docker | -      | -                      | Persiste <code>/var/lib/mysql</code> para que los datos sobrevivan a la recreación del contenedor.                      |

*Nota: el volumen y la red no son servicios, son recursos de apoyo. Por eso se habla de **tres servicios** y dos recursos adicionales.*

## 1.3 Diagrama de arquitectura

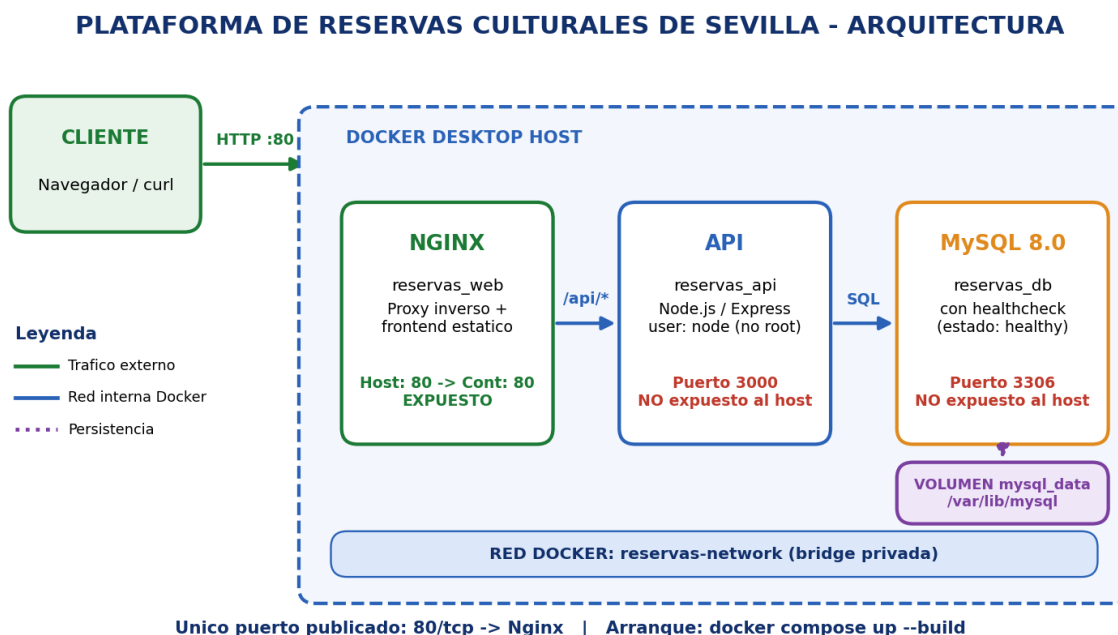


Figura 1. Arquitectura de la plataforma: cliente, Nginx, API, base de datos, red, puertos y volumen.

## 1.4 Flujo de una petición

1. El navegador (o `curl`) envía `GET http://localhost/api/eventos` al puerto 80 del host.
2. Docker reenvía el puerto 80 del host al contenedor `reservas_web` (Nginx).
3. Nginx comprueba la ruta: si empieza por `/api/`, la reenvía a `http://api:3000` por la red interna (`api` es el nombre del servicio, resuelto por el DNS de Docker).
4. La API Node.js abre una consulta SQL contra `db:3306` y obtiene los eventos.
5. La respuesta JSON recorre el camino inverso hasta el cliente.

El cliente solo conoce el puerto 80. API y base de datos son invisibles desde fuera de la red Docker.

## 2. Validación y despliegue de la infraestructura

### 2.1 Estructura del repositorio y Dockerfile de la API

El repositorio Git contiene todo lo necesario para una ejecución limpia en otro equipo: código, Dockerfiles, Compose, configuración y documentación.

```
proyecto-reservas/
├── api/
│   ├── Dockerfile
│   ├── index.js
│   └── package.json
├── db/
│   └── init.sql          # esquema y datos iniciales
├── nginx/
│   ├── Dockerfile
│   └── default.conf      # configuración del proxy
├── .dockerignore
├── .env.example          # plantilla SIN secretos reales (en Git)
├── .env                  # secretos reales (ignorado por Git)
└── .gitignore
```

```
└─ compose.yaml
└─ README.md
```

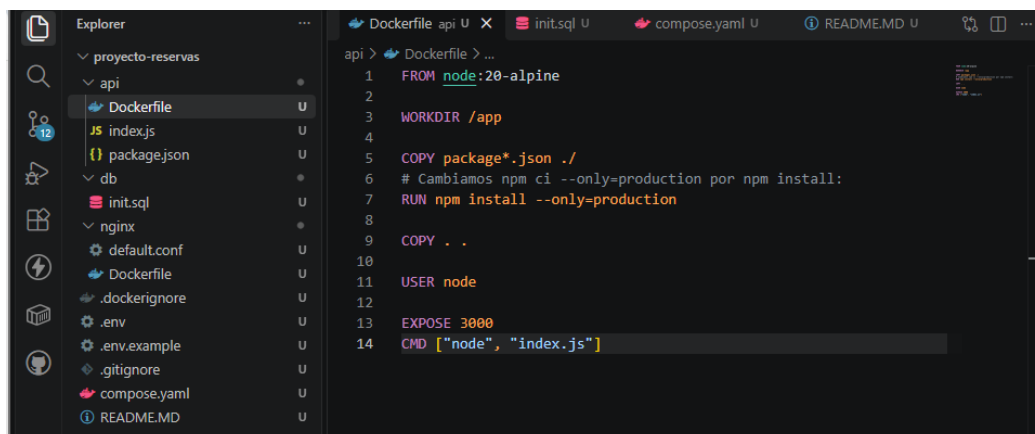


Figura 2. Estructura del proyecto en el editor y Dockerfile de la API.

Decisiones del Dockerfile de la API (cada línea tiene un motivo):

| Instrucción                               | Por qué                                                                                                                                                                                                                |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FROM node:20-alpine                       | Imagen <b>versionada</b> (no <code>latest</code> ) y ligera: el resultado es reproducible y con menos superficie de ataque.                                                                                            |
| WORKDIR /app                              | Directorio de trabajo fijo dentro del contenedor.                                                                                                                                                                      |
| COPY package*.json ./ antes que el código | Aprovecha la <b>caché de capas</b> : si solo cambia el código, no se reinstalan las dependencias.                                                                                                                      |
| RUN npm install --only=production         | Instala solo las dependencias de producción ( <b>dependencias controladas</b> ). Mejora posible: versionar <code>package-lock.json</code> y usar <code>npm ci --omit=dev</code> para builds exactamente reproducibles. |
| COPY . .                                  | Copia el código. El <code>.dockerignore</code> evita meter <code>node_modules</code> , <code>.env</code> o <code>.git</code> en la imagen.                                                                             |
| USER node                                 | La API <b>no se ejecuta como root</b> : si alguien explotara una vulnerabilidad, tendría privilegios mínimos.                                                                                                          |
| EXPOSE 3000 + CMD ["node", "index.js"]    | Documenta el puerto interno y arranca la aplicación. <code>EXPOSE</code> no publica el puerto en el host.                                                                                                              |

## 2.2 Paso 1: validar la configuración (docker compose config)

Antes de construir nada se valida el archivo `compose.yaml`. El comando comprueba la sintaxis, resuelve las variables de entorno y muestra el archivo final que Docker interpretará. Si hay errores (indentación, variable sin definir, servicio inexistente) los muestra y termina con error.

```
docker compose config
```

```
volumes:
  - type: volume
    source: mysql_data
    target: /var/lib/mysql
    volume: {}
  - type: bind
```

## 2.3 Paso 2: construir y levantar (docker compose up --build)

Con la configuración validada se construyen las imágenes de la API y de Nginx a partir de sus Dockerfile y se arrancan los tres contenedores. Con `-d` (detached) quedan en segundo plano:

```
docker compose up --build -d
```

```
#20 [api] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 7/7
  ✓ Image proyecto-reservas-api          Built          6.3s
  ✓ Image proyecto-reservas-web          Built          6.3s
  ✓ Network proyecto-reservas_reservas-network Created        0.0s
  ✓ Volume proyecto-reservas_mysql_data  Created        0.0s
  ✓ Container reservas_db                Healthy       16.0s
  ✓ Container reservas_api               Started       16.1s
  ✓ Container reservas_web               Started       16.3s
PS C:\Users\DAW2\Desktop\proyecto-reservas>
```

Figura 4. Construcción de imágenes y arranque: red y volumen creados, db en estado Healthy, api y web Started.

Se observa el **orden de arranque**: primero se crean la red y el volumen; después `reservas_db` pasa a **Healthy** (16.0 s) y solo entonces se inician `reservas_api` y `reservas_web`. Esto se consigue con el healthcheck de la base de datos y `depends_on` con `condition: service_healthy`:

```
# idea del compose (resumen)
api:
  depends_on:
    db:
      condition: service_healthy # la API espera a que MySQL esté lista
```

Sin esta condición, la API podría arrancar antes de que MySQL aceptara conexiones y fallar al conectarse.

## 2.4 Paso 3: comprobar el estado (docker compose ps)

```
docker compose ps
```

The screenshot shows a code editor with a README.md file open. The README.md file contains the following content:

```
1 # Plataforma de Reservas Culturales de Sevilla - Infraestructura Docker
2 ## Arquitectura del Sistema
3 - Nginx (reservas_web): servidor web y proxy inverso expone únicamente el puerto 80.
4 - **Node.js (reservas_api)**: Servicio API ejecutable en Node 20 en la red interna (puerto 3000).
5 - **MySQL (reservas_db)**: Base de datos relacional con comprobación de salud (healthcheck) y persistencia de datos montada en el volumen 'mysql_data'.
6
7 ## Requisitos
8 - Docker Desktop instalado y en ejecución.
9
10 ## Pasos para el Despliegue
11 1. Clonar el repositorio:
12    `` bash
13    git clone https://github.com/adrianlopez25-cmyk/proyecto-reservas.git https://github.com/adrianlopez25-cmyk/proyecto-reservas.git
14    cd proyecto-reservas
```

The terminal window shows the output of the command `docker compose ps`:

```
PS C:\Users\adria\Desktop\proyecto-reservas> docker compose ps
NAME                IMAGE                COMMAND                SERVICE        CREATED        STATUS        PORTS
reservas_api        proyecto-reservas-api  "docker-entrypoint.s..." api            27 minutes ago  Up 27 minutes  3000/tcp
reservas_db         mysql:8.0            "docker-entrypoint.s..." db             27 minutes ago  Up 27 minutes  3306/tcp
reservas_web        proyecto-reservas-web  "/docker-entrypoint..." web            27 minutes ago  Up 27 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp
```

Figura 5. Estado de los servicios: los tres en Up y db en (healthy).

| Servicio     | Estado esperado | Puertos            | Lectura                                                              |
|--------------|-----------------|--------------------|----------------------------------------------------------------------|
| reservas_web | Up              | 0.0.0.0:80->80/tcp | Accesible desde el navegador del host.                               |
| reservas_api | Up              | 3000/tcp           | Sin flecha ->: solo accesible dentro de la red Docker (desde Nginx). |
| reservas_db  | Up (healthy)    | 3306/tcp           | Sin flecha ->: solo accesible desde la API.                          |

Criterio de aceptación: **solo el puerto 80 aparece publicado** en el host. Es la prueba directa del requisito "la base de datos no debe publicar su puerto".

## 3. Verificación funcional: pruebas HTTP, aislamiento y logs

### 3.1 Prueba HTTP a través de Nginx

Se consulta la API desde el cliente. En PowerShell se usa `curl.exe` (porque `curl` es un alias de otro comando):

```
curl.exe -i http://localhost/api/eventos
```

Se espera **HTTP/1.1 200 OK** y un JSON con los eventos sembrados por `init.sql` (Taller de Cerámica Triana y Ruta Nocturna Barrio Santa Cruz). Esto confirma que la cadena **Nginx** → **Node.js** → **MySQL** funciona de extremo a extremo.

```

Content-Type: application/json; charset=utf-8
Content-Length: 376
Connection: keep-alive
X-Powered-By: Express
ETag: W/"178-XHDWMQ17leLdQIivf1isTh4Q0yE"

[{"id":1,"titulo":"Taller de Cer mica Triana","descripcion":"Taller artesanal tradicional","fecha":"2026-10-15T10:00:00.000Z"}, {"id":2,"titulo":"Ruta Nocturna Barrio Santa Cruz","descripcion":"Visita guiada de leyendas","fecha":"2026-10-16T21:00:00.000Z"}, {"id":3,"titulo":"Prueba Persistencia","descripcion":"Evento aadido manualmente","fecha":"2026-10-20T12:00:00.000Z"}]
PS C:\Users\adria\Desktop\proyecto-reservas>

```

### 3.2 Prueba de aislamiento

Se comprueba que API y base de datos **no** son alcanzables desde el host; solo debe responder el puerto 80:

```

Test-NetConnection localhost -Port 80      # TcpTestSucceeded : True
Test-NetConnection localhost -Port 3000    # TcpTestSucceeded : False
Test-NetConnection localhost -Port 3306    # TcpTestSucceeded : False
docker compose port db 3306                # sin salida: no hay puerto publicado

```

```

RemoteAddress      : ::1
RemotePort         : 3306
InterfaceAlias     : Loopback Pseudo-Interface 1
SourceAddress      : ::1
PingSucceeded      : True
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded   : False

PS C:\Users\adria\Desktop\proyecto-reservas>

```

### 3.3 Logs

Los logs evidencian el arranque y permiten diagnosticar incidencias:

```

docker compose logs --tail=30 db      # MySQL listo para conexiones
docker compose logs --tail=20 api     # API escuchando en el puerto 3000, sin errores de
conexión
docker compose logs --tail=20 web     # peticiones HTTP recibidas por Nginx
docker compose logs -f                # seguimiento en directo de los tres servicios

```

```

PS C:\Users\adria\Desktop\proyecto-reservas> docker compose logs --tail=20 api
reservas_api | API escuchando en el puerto 3000
PS C:\Users\adria\Desktop\proyecto-reservas> docker compose logs --tail=30 db
reservas_db | 2026-10-01 17:15:34+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
reservas_db | 2026-10-01 17:15:34+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
reservas_db | 2026-10-01 17:15:34+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
reservas_db | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
reservas_db | 2026-10-01T17:15:34.732420Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecate
ed and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
reservas_db | 2026-10-01T17:15:34.734961Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting a

```

### 3.4 Plan de pruebas y criterios de aceptación

| ID | Prueba                                                            | Criterio de aceptación                                   | Evidencia                |
|----|-------------------------------------------------------------------|----------------------------------------------------------|--------------------------|
| T1 | Configuración válida<br><code>docker compose config</code>        | Termina sin errores                                      | Figura 3                 |
| T2 | Arranque con un comando<br><code>docker compose up --build</code> | Imágenes construidas; db Healthy; api y web Started      | Figura 4                 |
| T3 | Estado de servicios<br><code>docker compose ps</code>             | Tres servicios Up; db healthy                            | Figura 5                 |
| T4 | Petición HTTP por Nginx<br><code>GET /api/eventos</code>          | HTTP 200 y JSON con los eventos de <code>init.sql</code> | Apartado 3.1 / Figura 10 |
| T5 | Aislamiento de puertos                                            | Puertos 3000 y 3306 no accesibles desde el host; solo 80 | Apartado 3.2             |
| T6 | Logs de arranque                                                  | Sin errores de conexión a la base de datos               | Apartado 3.3             |
| T7 | Persistencia tras <code>down + up</code>                          | El registro insertado a mano sigue existiendo            | Figuras 7 a 10           |

## 4. Prueba de persistencia de la base de datos (volumen `mysql_data`)

Requisito crítico: demostrar que los datos sobreviven a la **recreación** del contenedor de base de datos mientras se conserve el volumen. Un contenedor es efímero: su sistema de archivos se destruye con él. El volumen `mysql_data` vive fuera del ciclo de vida del contenedor y se monta en `/var/lib/mysql`, que es donde MySQL guarda sus datos.

### 4.1 Insertar un registro de prueba

Se añade un evento directamente en la base de datos en ejecución. Para **no escribir la contraseña en el comando** (ni en la memoria ni en el historial de la terminal), se usan las variables que ya existen dentro del contenedor:

```
@
INSERT INTO eventos (titulo, descripcion, fecha)
VALUES ('Prueba Persistencia', 'Verificando volúmenes', '2026-10-20 12:00:00');
"@ | docker exec -i reservas_db sh -c 'mysql --default-character-set=utf8mb4 -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE"'
```

Comprobación dentro de MySQL (ahora hay 3 filas):

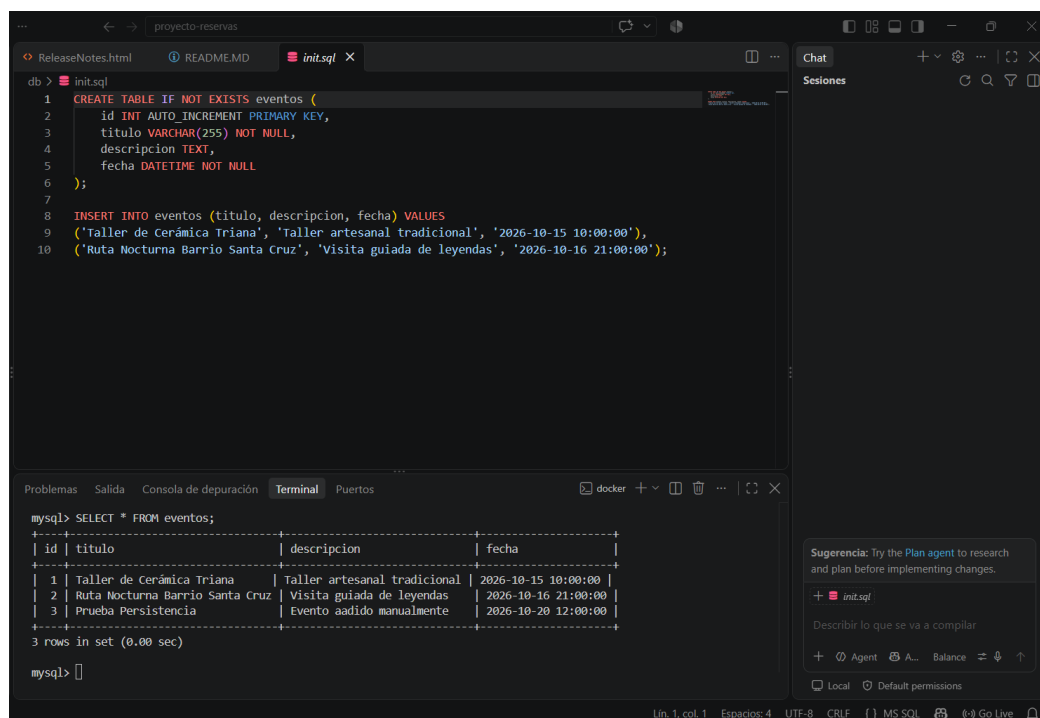


Figura 6. Tabla `eventos` con tres registros; el tercero (*Prueba Persistencia*) es el insertado a mano.

### 4.2 Destruir la infraestructura (`docker compose down`)

Se paran y eliminan los **tres contenedores y la red**. El volumen no se toca, porque `down` solo lo borra si se añade `-v`.

```
docker compose down
```

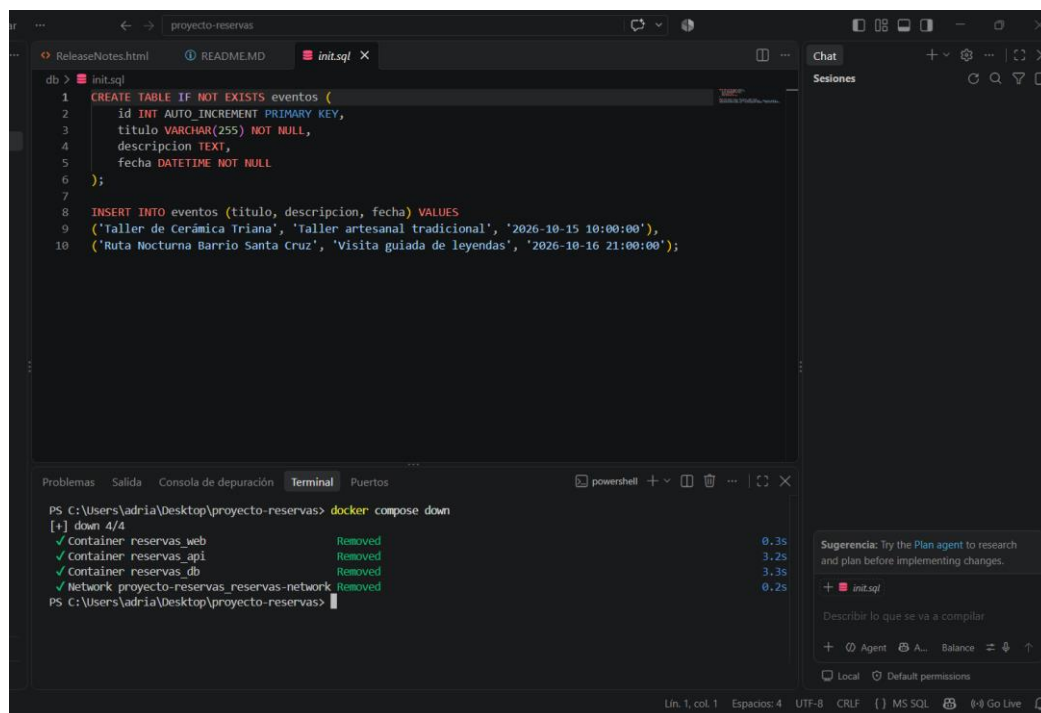


Figura 7. docker compose down: contenedores y red eliminados. El volumen no aparece en la lista.

### 4.3 Re-desplegar y verificar

```
docker compose up -d
```

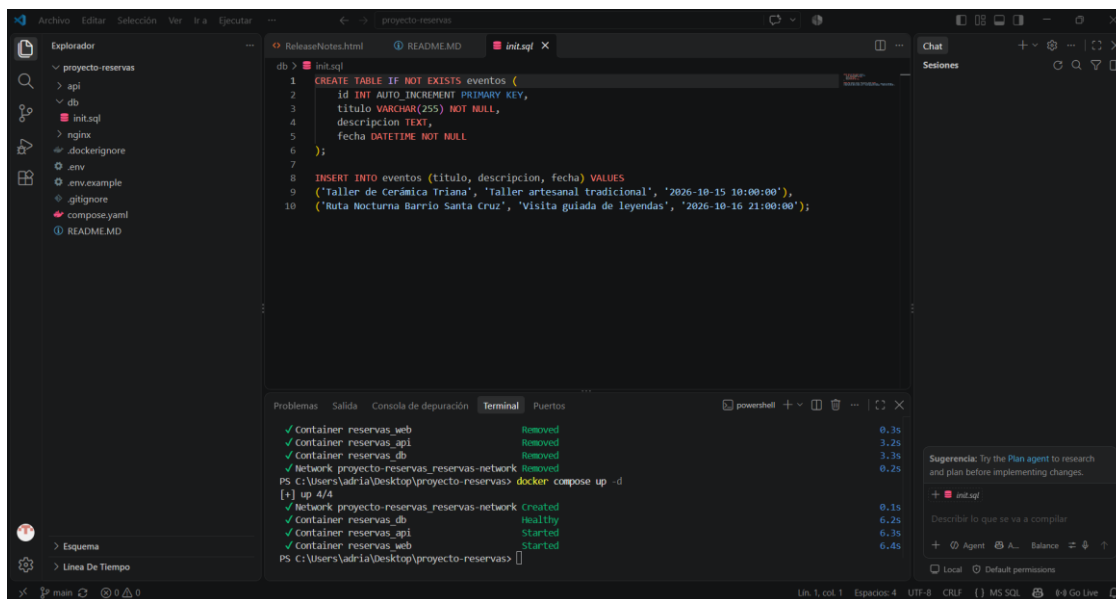


Figura 8. Re-despliegue desde cero: red y contenedores nuevos, db Healthy. No se crea volumen nuevo porque ya existe.

Al consultar de nuevo <http://localhost/api/eventos>, el tercer registro (**Prueba Persistencia**, id: 3) sigue existiendo:

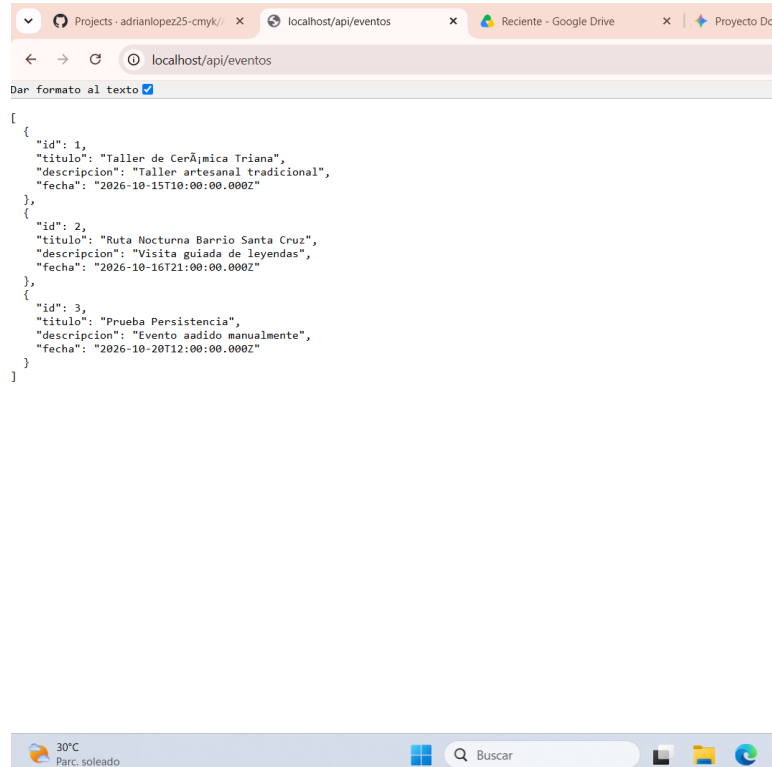


Figura 9. Respuesta de /api/eventos tras recrear los contenedores: el registro 3 se conserva.

## 4.4 Conclusión técnica

Los contenedores `reservas_db`, `reservas_api` y `reservas_web` fueron eliminados y creados de nuevo, pero el volumen `mysql_data` se conservó y se volvió a montar en `/var/lib/mysql`. MySQL encontró su directorio de datos ya inicializado y arrancó con la información previa. El registro insertado a mano permanece, lo que verifica el requisito de persistencia.

Detalle importante: el script `init.sql` solo se ejecuta cuando el directorio de datos está **vacío** (primer arranque). Por eso, tras recrear, los eventos 1 y 2 no se duplican. En Docker Desktop los volúmenes administrados residen dentro de la máquina virtual de Docker, no en una carpeta visible del sistema de archivos de Windows.

## 4.5 Contraprueba (opcional)

Para demostrar que la persistencia la aporta realmente el volumen y no otra cosa, se puede repetir la prueba con `-v`, que **elimina también el volumen**:

```
docker compose down -v
docker compose up -d
```

Resultado esperado: solo reaparecen los eventos 1 y 2 de `init.sql`; el registro 3 se ha perdido.

**Atención:** este comando borra los datos definitivamente; hazlo al final de las pruebas.

## 5. Seguridad y decisiones de diseño

### 5.1 Medidas aplicadas y su justificación

| Decisión                                                                                             | Justificación                                                                                    | Riesgo que reduce                                              |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Solo Nginx publica puerto (80)                                                                       | Un único punto de entrada controlado.                                                            | Acceso directo a API o BD desde fuera.                         |
| API y BD sin <code>ports</code> : hacia el host                                                      | Solo son accesibles por la red interna.                                                          | Ataques o accesos externos al puerto 3306/3000.                |
| Credenciales en <code>.env</code> (ignorado por Git) y <code>.env.example</code> sin secretos reales | Los secretos no se versionan; cada entorno define los suyos.                                     | Filtrado de contraseñas en el repositorio.                     |
| Imágenes versionadas ( <code>node:20-alpine</code> , <code>mysql:8.0</code> )                        | Builds reproducibles; el comportamiento no cambia por una actualización inesperada.              | Roturas por <code>latest</code> ; superficie de ataque grande. |
| <code>USER node</code> en la API                                                                     | Mínimo privilegio.                                                                               | Escalada de privilegios si se compromete la aplicación.        |
| <code>.dockerignore</code>                                                                           | Excluye <code>.env</code> , <code>.git</code> y <code>node_modules</code> del contexto de build. | Secretos o archivos innecesarios dentro de la imagen.          |
| <code>server_tokens off</code> en Nginx                                                              | Oculto la versión del servidor.                                                                  | Fingerprinting de versiones vulnerables.                       |
| Healthcheck + <code>depends_on: service_healthy</code>                                               | Orden de arranque fiable.                                                                        | API caída por conectarse antes de que la BD esté lista.        |
| <code>restart: unless-stopped</code>                                                                 | Reinicio automático ante fallos o reinicio de Docker.                                            | Indisponibilidad por caídas puntuales.                         |
| Volumen <code>mysql_data</code>                                                                      | Datos fuera del ciclo de vida del contenedor.                                                    | Pérdida de datos al recrear contenedores.                      |

### 5.2 Riesgos residuales y mejoras propuestas

- **HTTP sin cifrar.** El tráfico viaja en claro por el puerto 80. En producción habría que añadir TLS (HTTPS) en Nginx con un certificado válido.
- **Una única red.** Los tres servicios comparten `reservas-network`. Mejora: separar una red `frontend` (Nginx↔API) y otra `backend` (API↔BD, marcada como `internal`) para que Nginx no vea la base de datos.
- **Nginx como root.** La imagen oficial arranca el proceso maestro como root. Alternativa: `nginxinc/nginx-unprivileged` (escucha en 8080).
- **Gestión de secretos.** `.env` es suficiente para un entorno de laboratorio. En producción se usarían Docker secrets o un gestor de secretos.
- **Copias de seguridad.** Un volumen no es una copia de seguridad. Se recomienda un `mysqldump` periódico (apartado 7.3).
- **Dependencias.** Versionar `package-lock.json` y usar `npm ci --omit=dev`.

## 6. Buenas prácticas profesionales y problemas habituales

### 6.1 Lista de verificación antes de entregar

| Comprobación                                                                                                  | Cumple                   |
|---------------------------------------------------------------------------------------------------------------|--------------------------|
| <code>docker compose config</code> sin errores                                                                | <input type="checkbox"/> |
| Todo arranca con <code>docker compose up --build</code> desde un clon limpio                                  | <input type="checkbox"/> |
| Solo el puerto 80 publicado ( <code>docker compose ps</code> )                                                | <input type="checkbox"/> |
| <code>.env</code> en <code>.gitignore</code> ; solo <code>.env.example</code> en Git (con valores de ejemplo) | <input type="checkbox"/> |
| Dockerfile con imagen versionada y <code>USER</code> no root                                                  | <input type="checkbox"/> |
| <code>.dockerignore</code> presente                                                                           | <input type="checkbox"/> |
| Persistencia demostrada con <code>down + up</code>                                                            | <input type="checkbox"/> |
| Logs y pruebas HTTP documentados                                                                              | <input type="checkbox"/> |
| README con instalación, arranque, pruebas, incidencias y parada                                               | <input type="checkbox"/> |
| Capturas sin contraseñas visibles                                                                             | <input type="checkbox"/> |

### 6.2 Problemas habituales y diagnóstico

| Síntoma                                                | Causa probable                                                             | Diagnóstico y solución                                                                                                                                                                                                                                  |
|--------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>port is already allocated</code> al arrancar     | El puerto 80 lo usa otro programa (IIS, Apache, otro contenedor).          | <code>netstat -ano   findstr :80</code> . Parar el programa o cambiar a "8080:80" en <code>compose.yaml</code> .                                                                                                                                        |
| Nginx responde 502 Bad Gateway                         | La API no está arrancada o el <code>proxy_pass</code> apunta mal.          | <code>docker compose ps</code> y <code>docker compose logs api</code> . Comprobar que <code>proxy_pass</code> usa <code>http://api:3000</code> .                                                                                                        |
| La API da error de conexión a la BD                    | La BD no estaba lista, o las variables del <code>.env</code> no coinciden. | <code>docker compose logs db</code> y <code>logs api</code> . Revisar <code>depends_on: service_healthy</code> y los nombres de las variables.                                                                                                          |
| Cambié el <code>.env</code> y no se nota               | Los contenedores guardan las variables con las que se crearon.             | <code>docker compose up -d --force-recreate</code> .                                                                                                                                                                                                    |
| Cambié <code>init.sql</code> y no se aplica            | <code>init.sql</code> solo se ejecuta con el volumen vacío.                | <code>docker compose down -v</code> y <code>up</code> (borra los datos) o aplicar el SQL manualmente.                                                                                                                                                   |
| Caracteres raros (CerÃ¡mica, aadido)                   | Codificación: MySQL o el cliente no usan UTF-8.                            | Añadir <code>command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci</code> al servicio db y <code>SET NAMES utf8mb4;</code> al inicio de <code>init.sql</code> . Recrear con <code>down -v</code> para que se vuelva a ejecutar. |
| <code>npm ci</code> falla al construir                 | No hay <code>package-lock.json</code> en el contexto.                      | Versionar el lock o usar <code>npm install</code> .                                                                                                                                                                                                     |
| <code>error during connect</code> / Docker no responde | Docker Desktop no está iniciado.                                           | Abrir Docker Desktop y esperar al estado "running".                                                                                                                                                                                                     |

## 7. Parada y recuperación del servicio

### 7.1 Despliegue desde cero (para un tercer equipo)

1. Requisito: Docker Desktop instalado y en ejecución.
2. Clonar el repositorio y entrar en la carpeta: `git clone <URL>` y `cd proyecto-reservas`.
3. Crear la configuración local: `copy .env.example .env` (Windows) o `cp .env.example .env`, y editar las contraseñas.
4. Arrancar: `docker compose up --build`.
5. Verificar: abrir `http://localhost/api/eventos` y ejecutar `docker compose ps`.

### 7.2 Parada

| Comando                             | Efecto                                         | ¿Conserva los datos?        |
|-------------------------------------|------------------------------------------------|-----------------------------|
| <code>docker compose stop</code>    | Para los contenedores sin eliminarlos.         | Sí                          |
| <code>docker compose down</code>    | Elimina contenedores y red.                    | Sí (el volumen se conserva) |
| <code>docker compose down -v</code> | Elimina contenedores, red <b>y volúmenes</b> . | No: borra los datos         |

### 7.3 Copia de seguridad y restauración

```
# Copia de seguridad (PowerShell)
docker exec reservas_db sh -c 'mysqldump -u"$MYSQL_USER" -p"$MYSQL_PASSWORD"
"$MYSQL_DATABASE" | Out-File -Encoding utf8 backup.sql

# Restauración
Get-Content backup.sql -Raw | docker exec -i reservas_db sh -c 'mysql -u"$MYSQL_USER" -
p"$MYSQL_PASSWORD" "$MYSQL_DATABASE"'
```

### 7.4 Recuperación ante fallo

1. Identificar el servicio afectado con `docker compose ps`.
2. Leer sus logs: `docker compose logs --tail=50 <servicio>`.
3. Reiniciar solo ese servicio: `docker compose restart <servicio>`; si persiste, recrearlo: `docker compose up -d --force-recreate <servicio>`.
4. Si la base de datos está dañada, restaurar la última copia (apartado 7.3).

## Anexo. Archivos de configuración de referencia

**.env.example** (plantilla; en el repositorio solo valores de ejemplo, nunca la contraseña real):

```
DB_NAME=reservas_sevilla
DB_USER=cultural_user
DB_PASSWORD=SuperSecretPassWord2026!
DB_ROOT_PASSWORD=RootSuperSecret2026!
DB_HOST=db
PORT=3000
```

**compose.yaml** (esquema):

```
services:
  web:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: reservas_web
    restart: unless-stopped
    ports:
      - "80:80" # ÚNICO PUERTO EXPUESTO AL HOST
    networks:
      - reservas-network
    depends_on:
      - api

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: reservas_api
    restart: unless-stopped
    # No se exponen puertos al host por seguridad
    environment:
      - PORT=${PORT}
      - DB_HOST=${DB_HOST}
      - DB_NAME=${DB_NAME}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
    networks:
      - reservas-network
    depends_on:
      db:
        condition: service_healthy

  db:
    image: mysql:8.0
    container_name: reservas_db
    restart: unless-stopped
    # No se exponen puertos al host
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - reservas-network
    healthcheck:
```

```
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${DB_ROOT_PASSWORD}"]
    interval: 5s
    timeout: 5s
    retries: 5

networks:
  reservas-network:
    driver: bridge

volumes:
  mysql_data:
    driver: local
```

**nginx/default.conf** (esquema):

```
server {
    listen 80;
    server_name localhost;

    # Ocultar la versión de Nginx por seguridad
    server_tokens off;

    # Redirigir peticiones de /api/ al contenedor de la API
    location /api/ {
        proxy_pass http://api:3000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    #Frontend
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```